C Programming - Assignment 1 (CO1)

Task 8: Trace the Compilation Process

Grama Jyothi Rural Electricity Co-operative - billing program

Bloom's Level: K3 - Apply | Marks: 4

Student name	CH Mukesh reddy
Register number	192420032
Repository	CSA02-<RegisterNumber>-Assignment1 (folder: Task08)
GitHub link	_____
Compiler	GCC 13.3.0 (Ubuntu 24.04), x86-64
Program traced	the billing program of Task 9 (main.c, 76 lines), and its 4-file split version (Section 8)

Contents

- 1. Commands used
- 2. The four stages at a glance
- 3. Stage 1 - Preprocessing (gcc -E)
- 4. Stage 2 - Compilation to assembly (gcc -S)
- 5. Stage 3 - Assembly to an object file (gcc -c)
- 6. Stage 4 - Linking to an executable
- 7. Compile-time, link-time and run-time errors
- 8. Warnings from gcc -Wall -Wextra
- 9. Alternative concept: several .c files with a shared header
- 10. Conclusion

1. Commands used

Every output in this report was produced by running the commands below on the billing program (`main.c`). The program has no external files; it reads two numbers from the keyboard.

```
gcc -E main.c -o main.i          # 1. preprocess only
gcc -S main.c -o main.s          # 2. compile to assembly
gcc -c main.c -o main.o          # 3. assemble to an object file
gcc main.o -o billing            # 4. link to an executable
gcc -Wall -Wextra -std=c11 -o billing main.c    # normal build with warnings on
```

2. The four stages at a glance

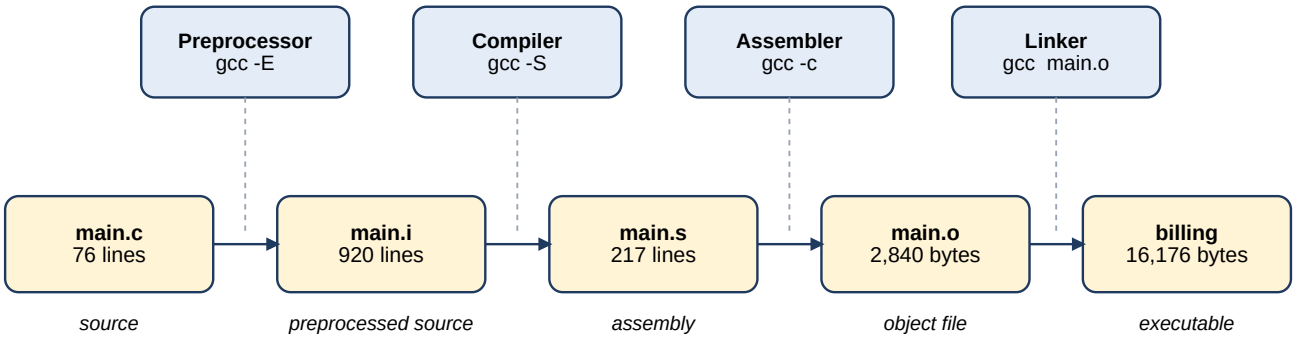


Figure 1: Each stage reads the file on its left and writes the file on its right. The sizes are measured from this program.

Stage	Command	Input	Output	Size	Readable?
Preprocess	gcc -E	main.c	main.i	920 lines (24,322 bytes)	Yes, still C
Compile	gcc -S	main.i	main.s	217 lines (3,719 bytes)	Yes, x86-64 assembly
Assemble	gcc -c	main.s	main.o	2,840 bytes (code 873 bytes)	No, binary ELF relocatable
Link	gcc main.o	main.o + C library	billing	16,176 bytes	No, binary ELF executable

3. Stage 1 - Preprocessing (gcc -E)

The preprocessor works on text only; it does not check C grammar. It does three things to `main.c`:

- **Expands #include.** The 76-line source becomes 920 lines. Almost all of the added lines come from `stdio.h` (42 linemarkers) and from `limits.h`, which is pulled in by the line `#include <limits.h>` even though this program does not use it. 152 lines of the output start with #: these are linemarkers that record which file and line each piece came from, so that error messages can point to the right place.
- **Replaces every macro by its value.** The names of the tariff constants no longer exist after this stage (table below).
- **Removes comments and the #define lines.** 15 lines of `main.c` contain a comment; the output contains none.

In main.c	In main.i	Why it matters
if (units > SLAB2_UNITS)	if (units > 200)	A tariff revision in the #define changes every use below it, in one place.
return FIXED_CHARGE_PAISE + energy;	return 5000LL + energy;	The LL suffix is kept, so the sum stays long long.
if (previous > METER_MAX || ...)	if (previous > 65535u || ...)	The u suffix keeps the comparison unsigned.
* RATE3_PAISE	* 450	No variable and no storage is used for the constant.

What was removed or replaced: the three #include lines (replaced by the contents of the headers), the seven #define lines (removed after their names were replaced by values), and all comments (removed). Running `gcc -E -P`, which also drops the linemarkers, leaves 349 lines, of which the program's own code is the last 45 lines, from the typedef onward.

4. Stage 2 - Compilation to assembly (gcc -S)

Now the compiler checks the grammar and types, and translates the C into assembly for the x86-64 processor. This is where compile-time errors are reported (Section 7). The function `consumption()` becomes the following (comments added by me; the compiler is run without optimisation, so the code is plain):

```
consumption:
    pushq   %rbp                    # set up stack frame
    movq    %rsp, %rbp
    movl    %edi, %edx              # first argument: previous
    movl    %esi, %eax              # second argument: current
    movw    %dx, -4(%rbp)           # store previous as 16 bits (movw = "move word")
    movw    %ax, -8(%rbp)           # store current as 16 bits
    movzwl  -8(%rbp), %eax          # load current, zero-extended
    subw    -4(%rbp), %ax           # 16-bit subtract: current - previous  <-- roll-over
    movzwl  %ax, %eax               # zero-extend the 16-bit result to 32 bits
    popq    %rbp
    ret
```

- The single instruction `subw` subtracts 16-bit values. For 65400 and 120 the result wraps inside the 16-bit register to 256, so no if-else is needed: the roll-over of the meter in Task 2 is the same wrap-around the processor performs.
- The compiler replaced three calls `printf("...\n")` that print a fixed text by calls to `puts` (for "Invalid input.", the range message and "Energy and fixed charges included."), because puts is simpler and faster when there is no format to process. The calls with %u and %lld still use printf.
- The assembly contains the calls `call consumption`, `call compute_bill`, `call __isoc99_scanf@PLT`, `call printf@PLT` and `call puts@PLT`. The names ending in @PLT are library functions whose code is not yet present.

5. Stage 3 - Assembly to an object file (gcc -c)

The assembler converts the assembly into machine code and writes it into `main.o` (2,840 bytes; 873 bytes of it are program code). The file is not yet runnable. `nm main.o` lists its symbols:

```
0000000000000000 T consumption       T = defined here, at offset 0x0
0000000000000021 T compute_bill                                  offset 0x21
0000000000000085 T main                                          offset 0x85
                 U printf            U = undefined: code lives in the C library
                 U puts
                 U __isoc99_scanf    (this is scanf)
                 U __stack_chk_fail  (inserted by the compiler's stack protection)
```

The three functions you wrote are already inside the object file, but their addresses are only offsets from the start of the file, and the library functions have no address at all. Filling these gaps is the job of the linker.

6. Stage 4 - Linking to an executable

The linker joins `main.o` with the start-up code and the C library, gives each function its final address, and resolves the U symbols. The result, `billing`, is 16,176 bytes (code 2,474, initialised data 624, uninitialised data 8). It is dynamically linked: printf, puts and scanf are not copied in but loaded from the shared C library when the program starts.

Symbol	In main.o (offset)	In billing (address)	printf / puts / scanf
consumption	0x0	0x11a9	Resolved to the shared C library at load time
compute_bill	0x21	0x11ca	
main	0x85	0x122e	

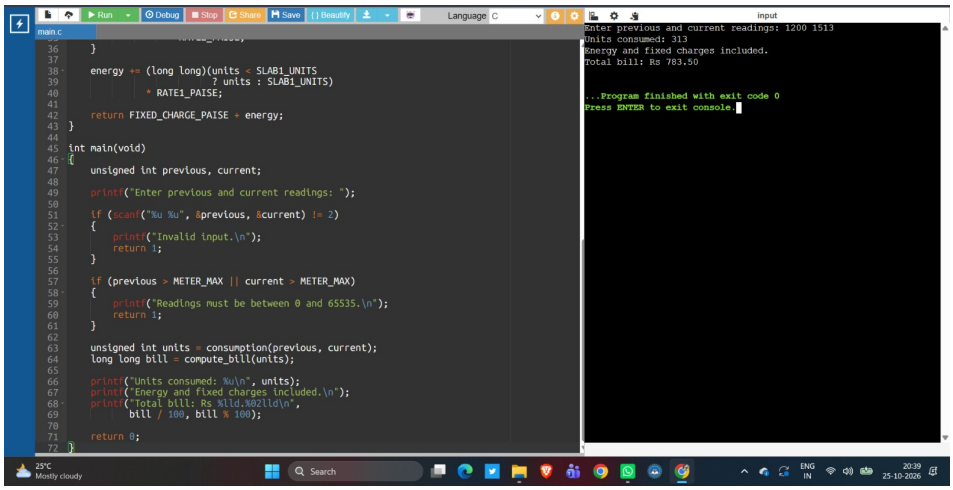


Figure 2: The linked billing program running in the online compiler. Input 1200 1513 gives 313 units and Rs 783.50, and the program ends with exit code 0.

7. Compile-time, link-time and run-time errors

To see the three kinds of error, one fault was introduced at a time into a copy of the billing program (the real program was never changed). The messages are copied from the compiler and the shell.

Kind	Fault introduced	Reported by	What happened
Compile-time	Removed the semicolon from `long long energy = 0;` in compute_bill	gcc (stage 2)	`error: expected ',' or ';' before 'if'` at line 32. No main.o is produced. The error is reported at the next statement (line 32), not at the line with the missing semicolon (line 31).
Link-time	compute_bill renamed to compute_bill_x in its definition, while main still calls compute_bill (declared by a prototype)	ld (stage 4)	The file compiles to an object file without error, then: `undefined reference to `compute_bill'` and `ld returned 1 exit status`. No executable is produced.
Run-time	`scanf("%u %u", previous, current)` without the & operators	operating system	It compiles and links (gcc only warns, Section 8). When run with input 1200 1513 the program stops with `Segmentation fault`, exit code 139, because scanf writes to the address held in the uninitialised variable.
Run-time	Division `total / slabs` where slabs is 0	operating system	`Floating point exception`, exit code 136 (an integer divide by zero). The compiler cannot know the value of slabs, so it accepts the code.

A misspelt function name such as `compute_bil(units)` shows the difference between versions of the compiler: GCC 13.3 (used here) accepts it with a warning `implicit declaration of function 'compute_bil'; did you mean 'compute_bill'?`, and the mistake would then appear later as a link-time error. Newer versions of GCC treat the same line as a compile-time error. This is why the warnings in Section 8 should never be ignored. **Rule of thumb:** compile-time errors stop stage 2 (grammar and types), link-time errors stop stage 4 (a name has no definition), and run-time errors appear only when the finished program runs with certain data.

8. Warnings from gcc -Wall -Wextra

The final program. `gcc -Wall -Wextra -Wpedantic -std=c11 -c main.c` prints no warnings. Adding `-Wconversion`, which is not part of -Wall or -Wextra, gives 2 warnings on line 67: the call `consumption(previous, current)` passes an unsigned int to a parameter of type meter_t (16 bits), so the value *may change*. This narrowing is deliberate, because main has already checked that both readings are at most 65535; writing `(meter_t)previous` would make the intention explicit and silence the warning.

An experiment with a faulty fragment. To see what the warnings look like, the fragment below was compiled with -Wall -Wextra. It is a reduced copy of compute_bill with four deliberate faults:

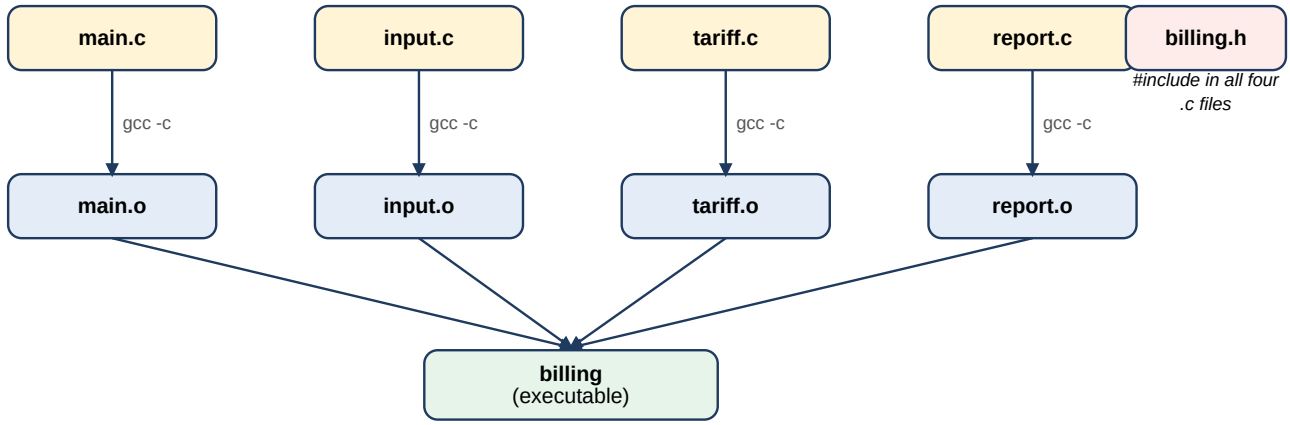
Line	Warning (flag)	What it tells me	How I resolved it
5	unused variable 'unused_slab' (-Wunused-variable)	A leftover declaration; the program logic never uses it.	Deleted the variable.
7	comparison of integer expressions of different signedness: 'int' and 'unsigned int' (-Wsign-compare)	`units > 200u` converts the signed units to unsigned; a negative value would become a huge positive one.	Compared with the plain int 200, so both sides are signed.
14	format '%d' expects 'int' but argument has type 'long long int' (-Wformat=), reported twice	bill / 100 and bill % 100 are long long; %d reads only an int and prints a wrong value (the Task 6 pitfall).	Used %lld and %02lld.

After these fixes the fragment compiles with 0 warnings. gcc gave no warning for `(units - 200) * RATE3_PAISE`, which is computed in int and could overflow for very large units; I added a cast to long long myself, as in the final program. **Lesson:** a clean -Wall -Wextra build means the compiler found nothing it can detect; it does not replace testing (Task 9).

9. Alternative concept and complexity analysis

Question. If the program were split into several .c files (input.c, tariff.c, report.c) with a shared header and compiled separately, instead of one file, what would the build process be?

Solution. The code is divided by responsibility. The tariff macros, the type `meter_t` and the four function prototypes move into `billing.h` (with an include guard); each .c file includes it. The build is controlled by a Makefile.



link step: gcc main.o input.o tariff.o report.o -o billing

Figure 3: Build structure of the split program. Every .c file includes billing.h, so a change to the header affects all four objects.

File	Contents	Lines
billing.h	include guard, tariff macros, meter_t, prototypes of read_readings, consumption, compute_bill, print_bill	19
input.c	read_readings(): prompt, scanf check, range check	20
tariff.c	consumption(), compute_bill()	25
report.c	print_bill()	9
main.c	main(): calls the four functions	13

```
billing: $(OBJS)                    # the executable depends on the four objects
    $(CC) $(OBJS) -o billing
%.o: %.c billing.h                  # each object depends on its .c file AND the header
    $(CC) $(CFLAGS) -c $< -o $@
```

The split program was built with `make` and gives the same results as the single file: 1200 1513 prints 313 units and Rs 783.50; 65400 120 prints 256 units and Rs 527.00; the letters abc print Invalid input; 70000 prints the range message. The behaviour of `make` after a change:

Change made	Commands run by make	Compilations
touch tariff.c (edit the tariff logic)	gcc -c tariff.c, then link	1 + link
touch billing.h (edit a tariff constant)	gcc -c for main, input, tariff, report, then link	4 + link
Single-file program, any change	gcc -c main.c, then link	1 (the whole program) + link

Measurements (median of 30 runs on the machine used for this report, in milliseconds; they depend on the computer, so repeat them on yours):

Build	Time (ms)	Object code (.o size)	Executable
Single file: compile main.c + link	33.6	2,840 bytes (code 873)	16,176 bytes
Split: change one .c file (1 compile + link)	31.0	four files: 1,944 + 1,720 + 1,888 + 1,448 = 7,000 bytes (code 343 + 222 + 376 + 253 = 1,194)	16,344 bytes
Split: full build from scratch (4 compiles + link)	78.5	same as above	16,344 bytes
Split: link step alone	17.1	-	-

Analysis. Let N be the total size of the source and n_i the size of the changed file.

- **Time after changing one file.** Single file: the whole program is compiled again, O(N), plus the link. Split: only the changed file is compiled, $O(n_i)$, plus the link, so it is much cheaper when N is large. A change to the header is the exception: all files that include it are compiled again, O(N) again.
- **Measured result here.** For this 76-line program the two builds take almost the same time (33.6 ms and 31.0 ms) because the time is dominated by starting gcc and the linker, not by the amount of code. The split build is actually slower from scratch (78.5 ms): it starts the compiler four times and reads stdio.h four times. The saving would appear only for programs with thousands of lines.
- **Space.** The split objects take 7,000 bytes against 2,840 bytes because every object file has its own headers and symbol tables, although the program code is only 321 bytes larger (1,194 against 873, mainly from the extra function-call set-up). The space is O(N) in both designs, with a larger constant for the split. The executable is 168 bytes larger.
- **Other trade-offs.** The split makes each file small enough to test and to share between students, and a change in the tariff is limited to tariff.c and billing.h. The single file is simpler for a program of this size and needs no Makefile.

10. Conclusion

- The 76-line source grew to 920 lines in the preprocessing stage, shrank to 217 lines of assembly, and became a 2,840-byte object file and a 16,176-byte executable. Each stage removed the information it no longer needed: comments and macro names first, then C syntax, then symbolic names for addresses.
- The roll-over calculation is carried out by one 16-bit machine instruction (`subw`), which shows why the unsigned 16-bit type in Task 2 behaves as it does.
- A missing semicolon is a compile-time error, a missing function body a link-time error, and a missing & in scanf a run-time error (segmentation fault); the last two are not found by the compiler alone.
- The final program builds without warnings under -Wall -Wextra -Wpedantic; -Wconversion shows two intentional narrowing conversions.
- Splitting into several files does not pay off in compile time for a program of this size, but it gives a clearer structure and would pay off as the program grows.

Files for the repository (folder Task08): main.c, main.i, main.s, main.o (or the commands to create them), err_compile.c, err_link.c, err_run.c, warn.c, the split/ folder with billing.h, input.c, tariff.c, report.c, main.c and Makefile, screenshots of each command, and the README.md entry.